Newton School of Technology

04

Lecture 04

Threads in Depth: Models and Implementations

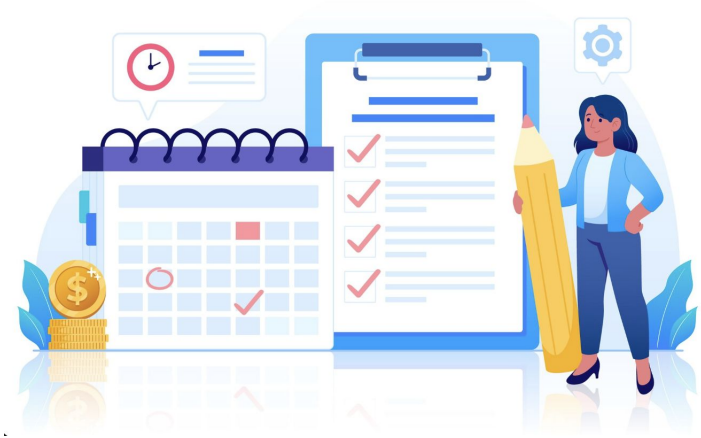
by - Brijesh Shukla

C07: Operating Systems

Today's Agenda

Newton School of Technology

- Introduction & The "Why”
- Foundations: Concurrency vs. Parallelism
- Deep Dive: Multithreading Models
- The Evolution of Execution
- Implementation: Linux & Java
- The Hazards



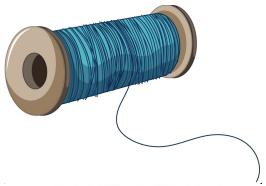
Process vs Thread

Newton School of Technology

A process is an independent program in execution with its own separate memory space.

- Its own memory address space (code, data, heap, stack)
- Its own system resources (file handles, network connections, etc.)
- At least one thread of execution
- Strong isolation from other processes

Creating a new process is relatively **expensive** because the **operating system** must allocate **separate memory and resources.**



A thread is a lightweight unit of execution within a process. Threads within the same process:

- Share the same memory address space and resources
- Have their own stack and program counter
- Can communicate easily through shared memory

Are **cheaper** to create and manage than processes

Analogy : The Housing Complex

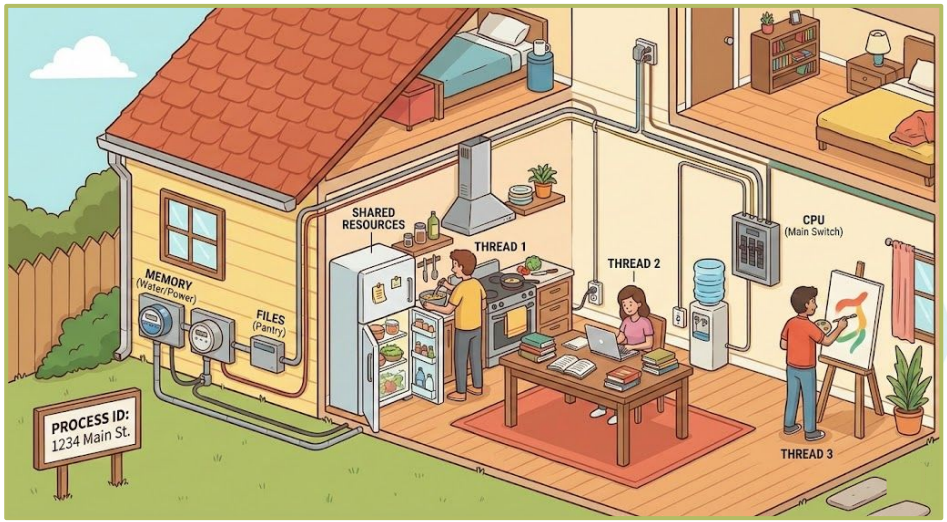
Newton School of Technology

The Computer: A giant city.

The Process: A single house. It has walls (**memory protection**), an address (**PID**), and resources (water/electricity = **file handles**). Building a house takes time.

The Thread: The people living inside the house.

The Point: If you want to get more work done (wash dishes + cook dinner), you don't build a second house (**Process fork**). You just get another person (**Thread**) in the kitchen. They share the fridge (**memory**), so they can communicate instantly without moving across the street (**IPC**).

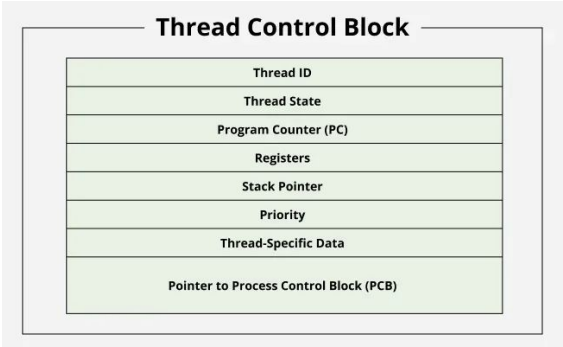


More on Threads

Newton School of Technology

What thread shares within same process?

- Code (text segment)
- Global variables (data segment)
- Heap memory
- Open file descriptors
- Signal handlers
- Process ID (PID)
- Working directory



What Each Thread Has Separately?

- Thread ID (TID)
- Program Counter (PC)
- Register set (including stack pointer)
- Stack (local variables, function call chain)
- Thread-specific data (errno, thread-local storage)

Checkpoint Question

Newton School of Technology

Why do threads need separate stacks but share the heap?

Each thread needs its own stack because the stack stores execution context - information that's unique to each thread's flow of control

The heap stores dynamically allocated data that threads often need to share

The stack is specific to a **thread's control flow**, whereas the heap is a **shared pool of memory** for dynamically allocated data that is common to all threads within a process.

```
void function_c() {
    int local_c = 3;
    // Thread 1 might be here
}

void function_b() {
    int local_b = 2;
    function_c();
}

void function_a() {
    int local_a = 1;
    function_b();  // Thread 2 might be here
}
```

Concurrency vs Parallelism

Newton School of Technology

Key Concept: Distinguishing structure from execution.

Concurrency is about dealing with lots of things at once.
Parallelism is about doing lots of things at once.

Analogy: The Coffee Shop

Concurrency (Single Core): One barista. There are two lines of customers. The barista takes an order from Line A, puts the cup under the machine, switches to Line B, takes payment, switches back to A. **They are making progress on both lines, but only working on one at a time.**

Parallelism (Multi-Core): Two baristas. Two machines. **Two customers get served at the exact same instant**

User Level Threads(ULT) - "Invisible" Threads

Newton School of Technology

The Concept: The OS is oblivious. The OS sees **one single process**. Inside that process, a Library (like the Java 1.1 JVM or a Go Runtime) acts as a **miniature OS**. It slices time and **switches between tasks manually.**

The Mechanics:

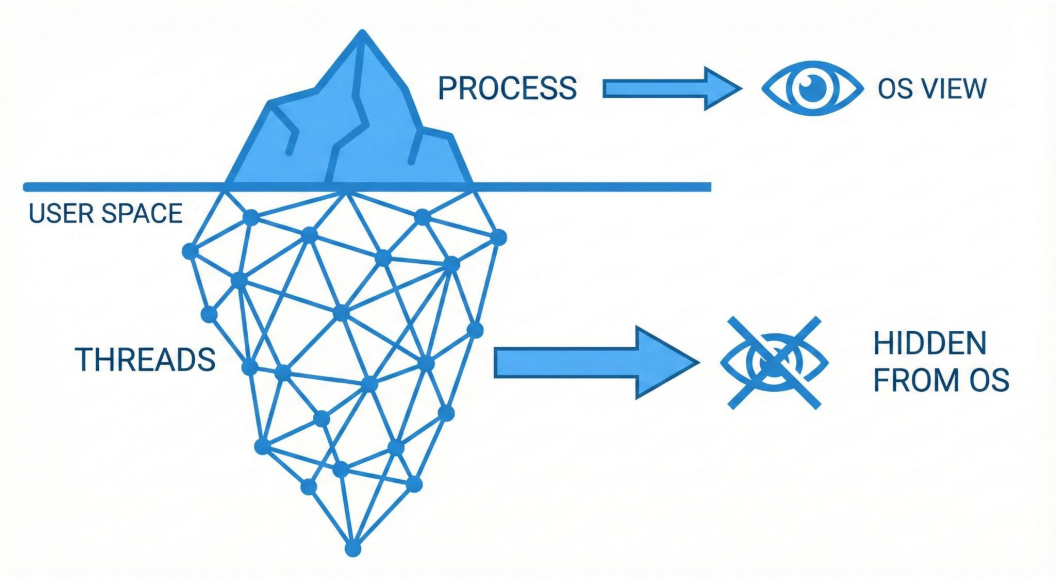
- Your code calls a function, and the Runtime saves the CPU registers to a variable in user memory (not kernel memory).
- The Runtime loads registers for the next thread.
- **Context Switch:** Extremely fast (nanoseconds) because no System Call is needed.

Also called "green threads" or "N:1 threads"

ULT – Fatal Flaw

Newton School of Technology

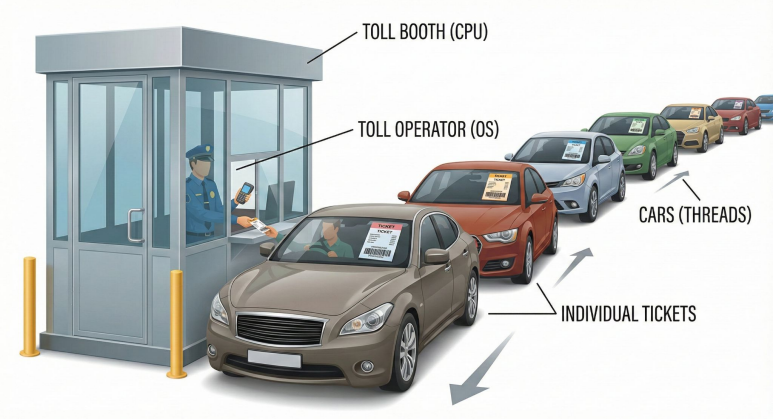
If User Thread A calls `read()` on a hard drive, the **Operating System** pauses the **entire process** because the OS doesn't know there are **other threads inside waiting to run.** The whole application freezes.



Kernel Level Threads – The "Heavy" Threads

Newton School of Technology

The Concept: The OS is fully aware. Every time you type new Thread(), the OS allocates a kernel structure (task_struct in Linux).



The Mechanics:

- Scheduling is handled by the OS Scheduler (e.g., CFS in Linux).
- **Context Switch:** Expensive (microseconds). Requires a "Mode Switch" (User Mode -> Kernel Mode), flushing caches, and security checks.
- **The Superpower:** True Parallelism. If Thread A blocks on I/O, the OS simply switches to Thread B.

Also known as: 1:1 Threads, Native Threads.

Comparison Table

Newton School of Technology

Feature	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Managed By	User-space Library	Operating System
Speed	Blazing Fast (Instruction cycles)	Slow (System Calls)
Blocking	Blocks entire process	Blocks only the specific thread
Multicore?	No (Usually runs on 1 core)	Yes (OS distributes across cores)
Example	Java 1.1, Python asyncio (conceptually)	Modern Java, C++ std::thread

Hybrid Threads – M:N (Many to Many)

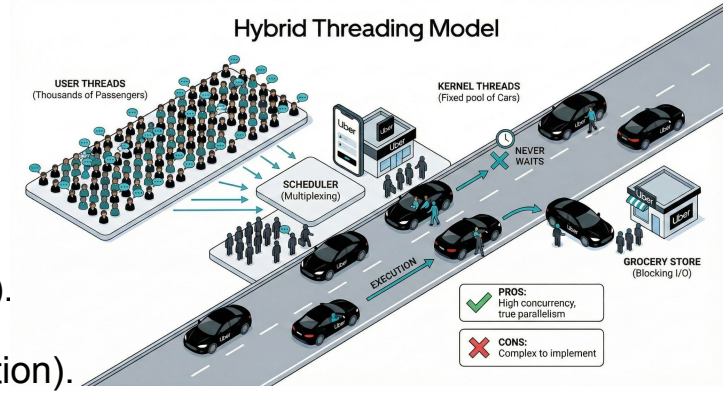
Newton School of Technology

The Concept: A pool of user threads is multiplexed onto a smaller pool of kernel threads.

Analogy: Uber/Ride-Sharing.

Passengers: User Threads (Thousands).

Cars: Kernel Threads (Fixed number, e.g., equal to CPU cores).



Logic: A car picks up a passenger, drives them for a bit (execution). If the passenger needs to stop for groceries (blocking I/O), they get out of the car. The car immediately picks up the next passenger. The car never waits.

Pros: Best of both worlds. High concurrency (millions of threads) with true parallelism.

Cons: Extremely complex to implement. The User Scheduler and Kernel Scheduler must coor

Virtual Threads (Java 21+)

Newton School of Technology

Virtual Threads are a modern, production-ready implementation of the M:N Model.

How it Works (Mounting/Unmounting):

Mounting: When a Virtual Thread needs to run, the JVM(Java Virtual Machine) assigns it to a "Carrier Thread" (a real Kernel Thread).

Running: The code executes on the CPU.

Unmounting (The Magic): When the code hits a blocking call (e.g., db.query()), the JVM detaches the Virtual Thread from the Carrier Thread. The Virtual Thread sits in heap memory (like a standard object), while the Carrier Thread grabs a new Virtual Thread to run.

Why it matters:

Scale: You can create 10,000,000 virtual threads on a standard laptop.

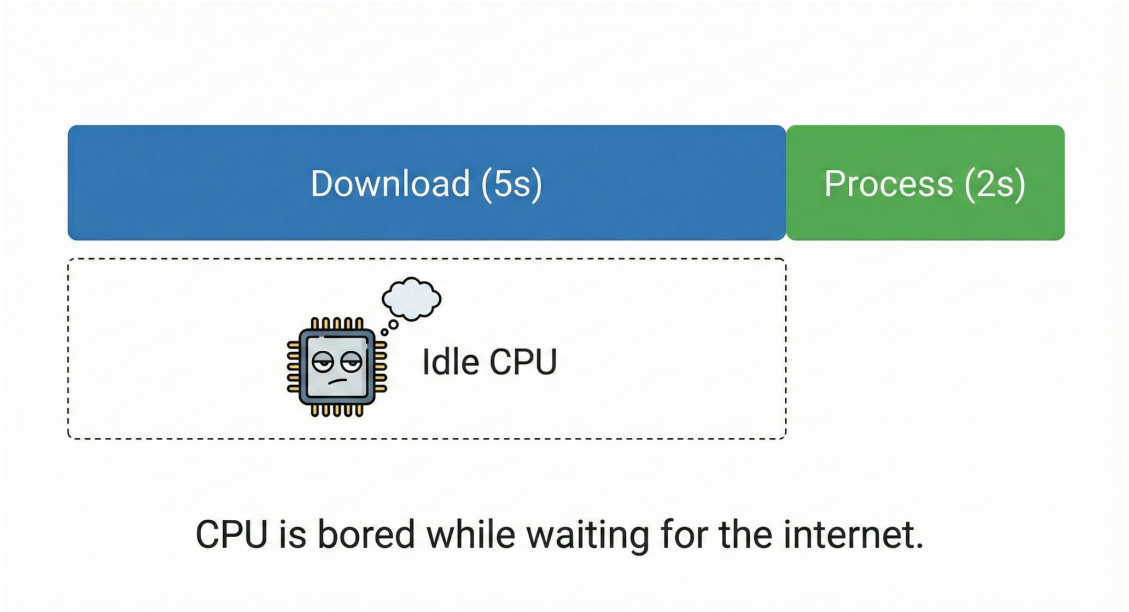
Style: You write simple, "blocking" style code (easy to read), but get the performance of complex asynchronous code.

Phase 1 – Sequential Execution

Newton School of Technology

Program WITHOUT Threads (Sequential)

```
void main() {
   downloadFile("A.txt"); // Blocks
for 5s
   processFile("A.txt");  // Runs
for 2s
   // Total Time: 7s. CPU is idle
during download.
}
```



Sequential Execution

Newton School of Technology

```c
#include <stdio.h>
#include <unistd.h>

void downloadFile(char* filename) {
    printf("Starting download: %s\n", filename);
    sleep(5); // Blocks for 5 seconds
    printf("Download complete: %s\n", filename);
}

void processFile(char* filename) {
    printf("Processing: %s\n", filename);
    sleep(2); // Runs for 2 seconds
    printf("Processing complete: %s\n", filename);
}

int main() {
    downloadFile("minishell.txt"); // Blocks for 5s
    processFile("A.txt");           // Runs for 2s
    // Total Time: 7s. CPU is idle during download.
    printf("Total time: 7s\n");
    return 0;
}
```

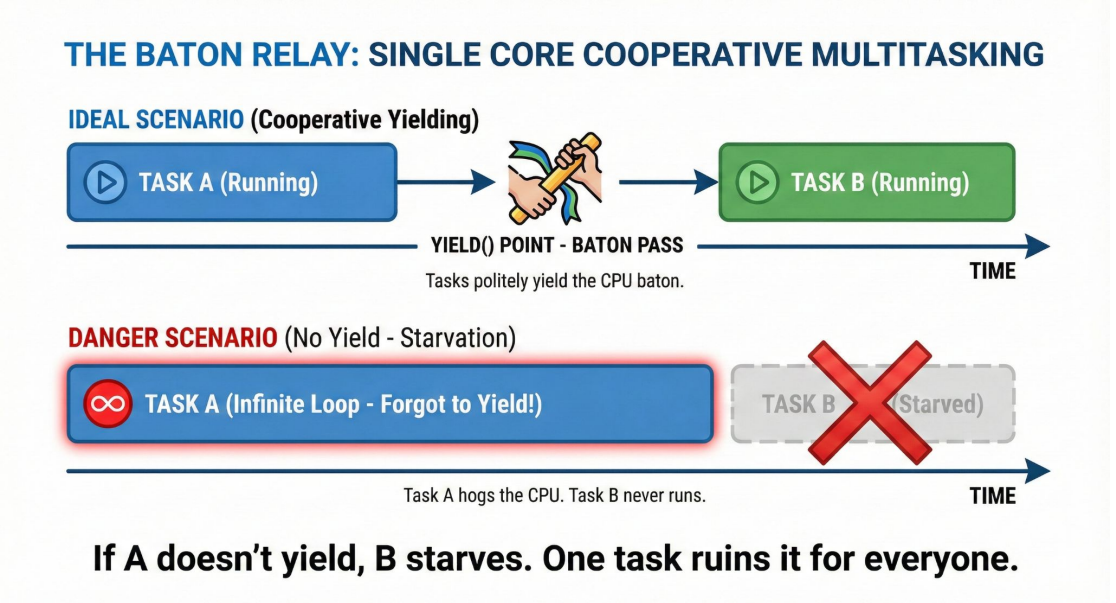
Phase 2 – User-Level / Cooperative (The Illusion)

Newton School of Technology

Concept: The tasks voluntarily "yield" control. This is how early concurrency worked. It is efficient but dangerous if a task refuses to yield.

```
void taskA() {
    startDownload("A.txt");
    yield(); // "I'm waiting, let someone else run!" → Switches to Task B in User Space
    // Resume here when download finishes
    processFile("A.txt");
}

void taskB() {
    calculateMath(); // Runs while Task A is waiting for download
}
```



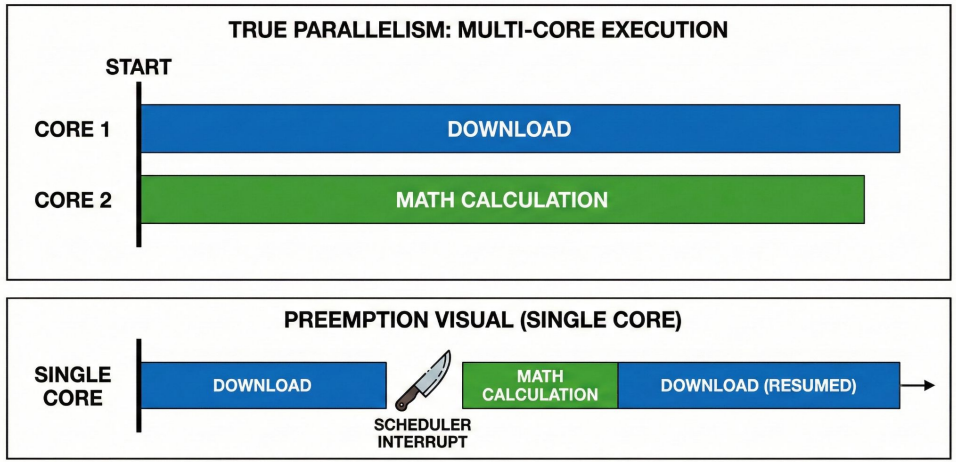
Phase 3 – Kernel Level (The Reality)

Newton School of Technology

By leveraging real OS threads, the scheduler handles context switching between tasks automatically. This removes the requirement for manual yielding.

```
void main() {
    Thread t1 = new Thread(() → {
        downloadFile("A.txt"); // Blocks THIS
thread, but OS switches to t2
    });
      Thread t2 = new Thread(() → {
        calculateMath(); // Runs on a
separate CPU Core
    });

    t1.start();
    t2.start();
}
```

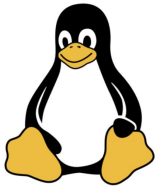


True Parallelism. The OS is the boss, not the code.

Implementation Details: Linux & Java

Newton School of Technology

Key Concept: How does the theory apply to real OSs and Languages?



The Revelation: "In Linux, there is no such thing as a thread."

Linux uses task_struct. Whether it's a process or a thread depends on flags passed to `clone()`.

`fork()` = `clone()` with no sharing.

NPTL (Native POSIX Thread Library):the modern standard (1:1 model).

JDK 1.0: Green Threads (N:1).

JDK 1.2 - 19: Native Threads (1:1). Stable, but heavy.

JDK 21 (Project Loom): Virtual Threads (M:N).

History repeats itself. We realized 1:1 was too heavy for modern web scale, so we went back to M:N, but this time with better hardware support.

The Dangers: Hazards & Synchronization

Newton School of Technology

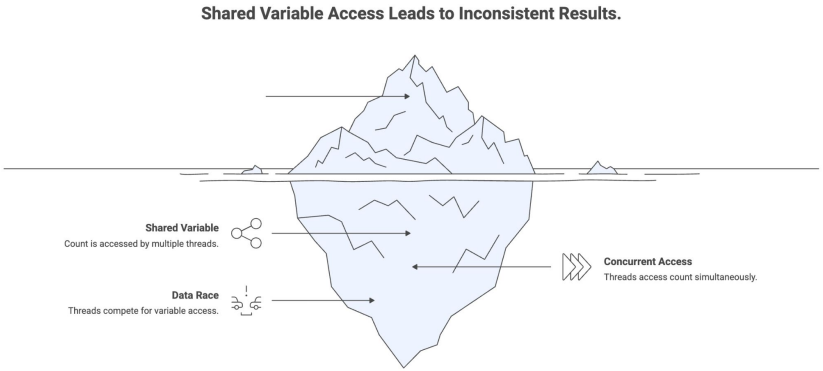
With great power comes great race conditions.

Scenario: count++.

The Breakdown: It's not one instruction. It's three:

1. Read count from memory to register.
2. Add 1 to register.
3. Write register back to memory.

The "Lost Update": If two threads do this at once, they might both read "5", both add 1, and both write "6". The count should be 7.



The Dangers: Hazards & Synchronization

Newton School of Technology

Deadlock: The Traffic Jam

Analogy: Four cars at a 4-way stop.

Rule: "Yield to the right."

The Problem: If 4 cars arrive simultaneously, everyone waits for the person on their right. No one moves. Forever.

The Technical Fix: Lock Ordering (Always acquire Lock A before Lock B).

Pthread API

Newton School of Technology

Pthreads() - POSIX Threads, is a standard API for creating and managing threads in C and C++ for concurrent execution

Category	Primary Types/Functions	Purpose
Management	`pthread_t`, `pthread_create`, `pthread_join`, `pthread_exit`	Creating, detaching, and joining threads.
Attributes	`pthread_attr_t`, `pthread_attr_init`	Setting stack size, scheduling policy, or detached state.
Synchronization	`pthread_mutex_t`, `pthread_cond_t`	Coordinating thread access to shared resources.
Thread Local Storage	`pthread_key_t`	Managing data unique to a specific thread.

Lifecycle Operations – Create

Newton School of Technology

pthread_create() - Spawning a Thread

This function starts a new thread in the calling process.

```
int pthread_create(
   pthread_t *thread,            // Output: The opaque thread ID
   const pthread_attr_t *attr,   // Attributes (Pass NULL for default)
   void *(*start_routine)(void*),// The function the thread will execute
   void *arg                     // Single argument passed to the function
);
// Returns 0 on success, error code on failure
```

Important: The function signature must be: `void* function_name(void* arg)`

Lifecycle Operations | Join | Detach

Newton School of Technology

`pthread_join()` - Waiting for Completion

Similar to wait() for processes, this blocks the calling thread until the target thread terminates.

```
int pthread_join(
   pthread_t thread,    // The specific thread ID
to wait for
   void **retval        // Output: Catches the
return value (void*)
);
// Must be called for joinable threads to release
resources (prevent "zombie threads")
```

`pthread_detach()` - Fire and Forget

If you do not need to wait for a thread or check its return value, you can detach it. The system automatically reclaims resources when the thread exits.

```
int pthread_detach(pthread_t thread);
// Note: You cannot join a thread once it
has been detached.
```

Lifecycle Operations | Termination

Newton School of Technology

`pthread_exit()` - Termination

Explicitly terminates the calling thread.

```
void pthread_exit(void *retval);
// The 'retval' is accessible to the thread
calling pthread_join()
// Note: Returning from the start_routine is
implicitly the same as calling this.
```

Let’s Implement

Newton School of Technology

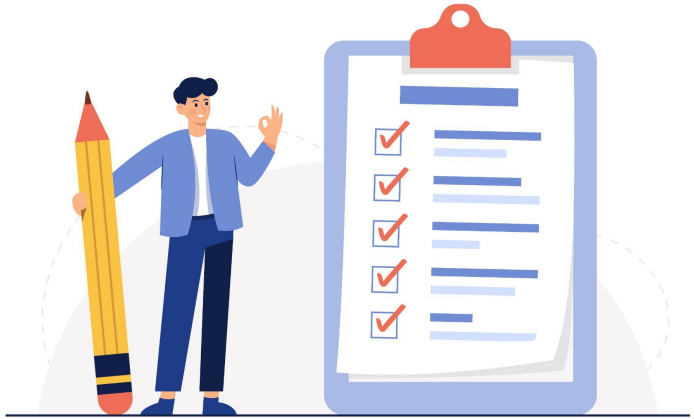
Create 3 threads, pass them a unique ID, simulate work, and retrieve a result.

Playground Link

Summary

Newton School of Technology

- Threads share memory, Processes don't.
- Concurrency is structure, Parallelism is execution.
- We moved from N:1 -> 1:1 -> M:N (Virtual Threads).
- Synchronization is the price we pay for shared state.



Thank You!

Newton School of Technology

Thank you for your attention. Feel free to ask questions or revisit the topics covered.